V1 MLP Architecture Sweep Results

LHCb Neural Network Track Extrapolation

Cost Function Weight Derivation for V2 Training

G. Scriven

April 2026

Abstract

We present the results of a systematic architecture sweep of 17 multi-layer perceptron (MLP) models trained to approximate the Runge–Kutta track extrapolator used in the LHCb experiment. Models were trained on 5M samples from a 50M-track dataset with variable propagation distance $\Delta z \in [100, 10\,000]$ mm and log-uniform momentum sampling $p \in [1, 100]$ GeV. The best model (`mlp_3x64`, 9 028 parameters) achieves a mean position error of 1.02 mm, but analysis reveals that 78.4% of the MSE loss is absorbed by the x component alone. Using the measured residual target variances, we derive inverse-variance cost function weights for V2 training that equalise gradient contributions across all four output components.

Contents

1 Introduction **2**

2 Experimental Setup **2**
2.1 Training Data . . . 2
2.2 Model Architecture . . . 2
2.3 Training Configuration . . . 2

3 Architecture Sweep Results **2**
3.1 Training Convergence . . . 3
3.2 Model Rankings . . . 4
3.3 Key Observations . . . 5

4 Diagnostic Analysis: Why V1 Performance Saturates **5**
4.1 MSE Contribution Breakdown . . . 5
4.2 Target Dynamic Range . . . 5
4.3 Momentum Dependence . . . 6
4.4 Spatial Dependence . . . 7
4.5 Error Distributions . . . 9

5 Cost Function Weights for V2 Training **10**
5.1 Residual Target Formulation . . . 10
5.2 Inverse-Variance Weighting . . . 10
5.3 Practical Weight Prescription . . . 11
5.4 Additional Architecture Recommendations . . . 11

6 Best Model Detailed Performance **11**

7 Summary **12**

1 Introduction

The LHCb track extrapolation system propagates charged particle states through the dipole magnetic field using numerical Runge–Kutta (RK) integration. The track state vector is $\mathbf{s} = (x, y, t_x, t_y, q/p)$ at longitudinal position z, where $t_x = dx/dz$, $t_y = dy/dz$ are direction tangents and q/p is the signed inverse momentum.

Neural network (NN) replacements offer the potential for significant speedup at the cost of some accuracy. In the V1 sweep, we train pure MLP models with SiLU activation to map:

$$f_\theta : (x, y, t_x, t_y, q/p, \Delta z) \mapsto (x', y', t'_x, t'_y) \tag{1}$$

using supervised MSE loss against RK4 ground truth. The key question is whether a feedforward network can learn the momentum-dependent field-driven deflection from raw state-to-state mapping.

2 Experimental Setup

2.1 Training Data

The dataset contains 50M track state pairs generated by a Python RK4 integrator (5 mm fixed steps) using trilinear interpolation on the LHCb field map (`twodip.rtf`, $81 \times 81 \times 146$ grid, 100 mm spacing). Each sample consists of:

- **Input**: $(x, y, t_x, t_y, q/p, \Delta z)$ — 6D state at origin z
- **Output**: (x', y', t'_x, t'_y) — 4D state at $z + \Delta z$

Key sampling parameters:

- Momentum: log-uniform $p \in [1, 100]$ GeV (equal weight per decade)
- Propagation: uniform $\Delta z \in [100, 10\,000]$ mm
- Spatial: z_{start} uniform in $[0, 14\,000]$ mm, positions within acceptance
- Charge: equal $q = \pm 1$ sampling

A 5M-sample subsample was used for training (80%/10%/10% train/val/test split, seed 42).

2.2 Model Architecture

All models are standard feedforward MLPs:

$$\mathbf{h}_0 = \frac{\mathbf{x} - \boldsymbol{\mu}_{\text{in}}}{\boldsymbol{\sigma}_{\text{in}}}, \quad \mathbf{h}_{l+1} = \text{SiLU}(\mathbf{W}_l \mathbf{h}_l + \mathbf{b}_l), \quad \hat{\mathbf{y}} = \boldsymbol{\sigma}_{\text{out}} \odot (\mathbf{W}_L \mathbf{h}_L + \mathbf{b}_L) + \boldsymbol{\mu}_{\text{out}} \tag{2}$$

where $\text{SiLU}(x) = x \cdot \sigma(x)$ is applied element-wise to all hidden layers, and z-score normalisation uses dataset-level statistics.

2.3 Training Configuration

- **Loss**: Mean squared error (MSE) on raw 4D output, equal weighting
- **Optimiser**: AdamW (lr $= 10^{-3}$, weight decay 10^{-4})
- **Scheduler**: Cosine annealing with 5-epoch linear warmup
- **Regularisation**: Gradient clipping at norm 1.0, early stopping (patience 30)
- **Batch size**: 4096
- **Infrastructure**: HTCondor GPU cluster (NVIDIA)

3 Architecture Sweep Results

Seventeen MLP architectures were trained, spanning 2–4 hidden layers and widths from 64 to 1024 neurons. All 17 models completed training via early stopping, with best epochs ranging from 22 to 169.

3.1 Training Convergence

Figure 1 shows the training and validation loss curves for all 17 models.

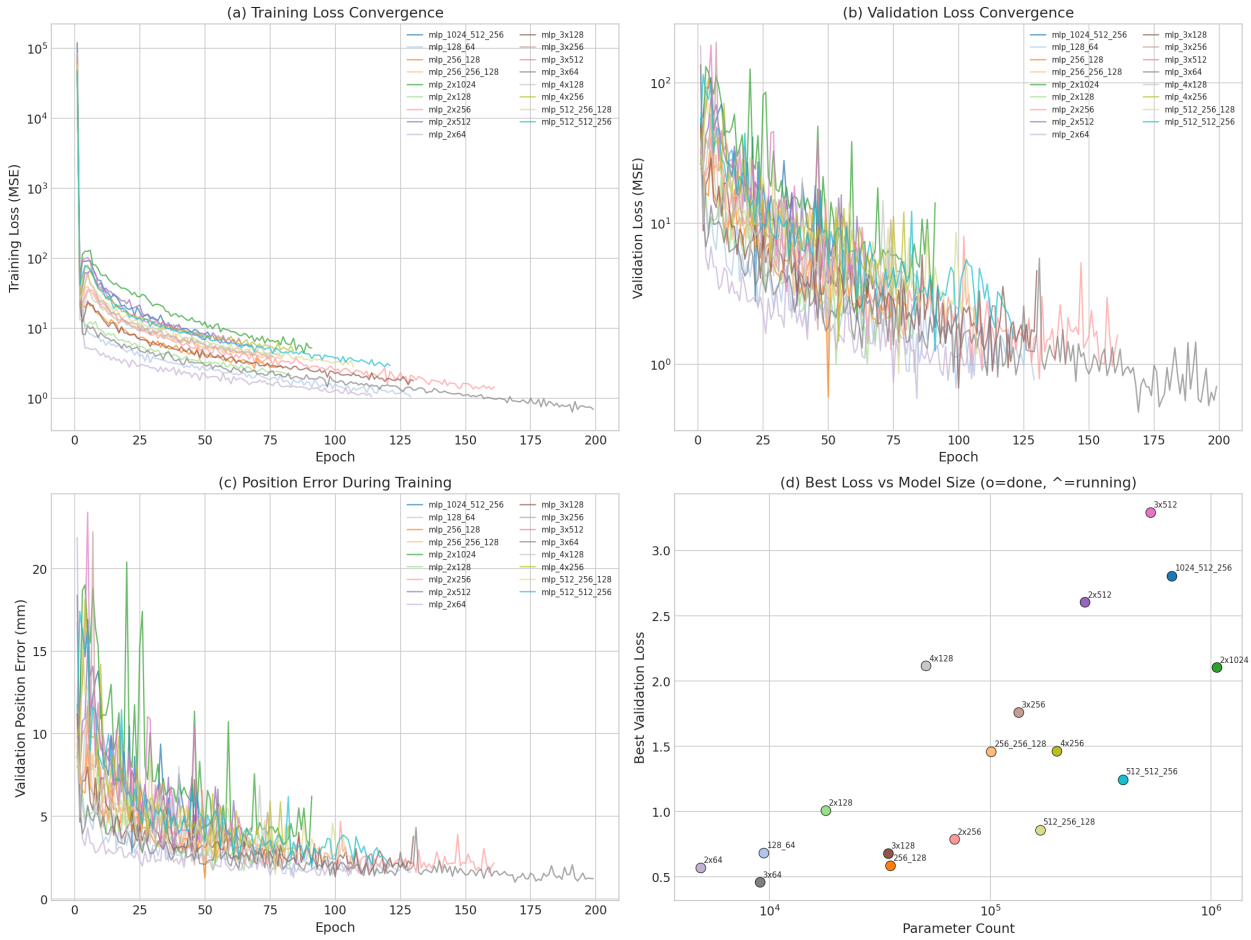

Figure 1: Training and validation loss curves for all V1 MLP architectures. All 17 models reached convergence via early stopping.

Figure 2 provides additional training diagnostics including learning rate schedules and gradient norms.

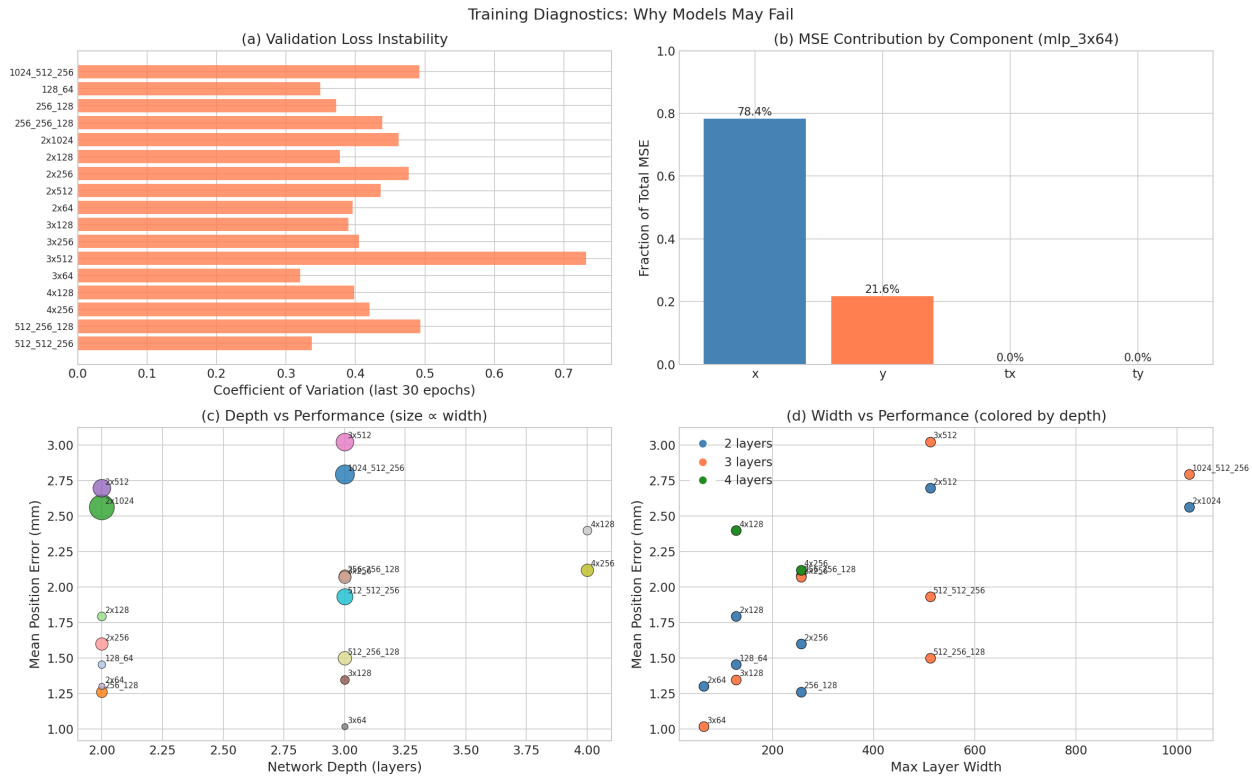

Figure 2: Training diagnostics: learning rate schedules and gradient norm evolution across epochs.

3.2 Model Rankings

Table 1 ranks all 17 models by mean position error on the 500K test set.

Table 1: Performance of all V1 MLP models, ranked by mean position error. Position errors in mm; slope errors are dimensionless.

Model	Layers	Params	Pos. mean (mm)	Pos. 95% (mm)	Pos. 99% (mm)	$\lvert t_x \rvert$ MAE	$\lvert t_y \rvert$ MAE
`mlp_3x64`	3	9 028	1.017	2.969	4.303	0.004041	0.004873
`mlp_256_128`	2	35 204	1.263	2.896	4.181	0.002319	0.002274
`mlp_2x64`	2	4 868	1.302	2.630	3.573	0.001662	0.001633
`mlp_3x128`	3	34 436	1.349	3.242	4.944	0.002441	0.002423
`mlp_128_64`	2	9 412	1.455	2.897	3.866	0.002845	0.002234
`mlp_512_256_128`	3	168 324	1.499	3.599	5.611	0.003355	0.003319
`mlp_2x256`	2	68 612	1.601	2.954	3.509	0.001858	0.001363
`mlp_2x128`	2	17 924	1.793	3.417	4.462	0.001567	0.001480
`mlp_512_512_256`	3	398 596	1.934	3.987	5.220	0.003418	0.002534
`mlp_3x256`	3	134 404	2.071	5.568	7.760	0.002595	0.002500
`mlp_256_256_128`	3	100 996	2.083	4.413	6.148	0.004859	0.003643
`mlp_4x256`	4	200 196	2.121	4.232	5.386	0.003433	0.003260
`mlp_4x128`	4	50 948	2.400	5.728	8.478	0.004519	0.005875
`mlp_2x1024`	2	1 060 868	2.562	5.055	6.727	0.002207	0.002017
`mlp_2x512`	2	268 292	2.697	5.887	9.273	0.001654	0.001833
`mlp_1024_512_256`	3	664 324	2.796	6.101	9.522	0.002769	0.002050
`mlp_3x512`	3	530 948	3.024	6.569	9.603	0.002870	0.002574

3.3 Key Observations

1. **Small models win**: The best model (`mlp_3x64`, 3 layers, 9K params) outperforms all larger architectures. The top 5 models all have ≤ 35K parameters, with `mlp_2x64` (4 868 params) ranking third.
2. **Diminishing returns from width**: Increasing layer width beyond 128 neurons provides no benefit and often degrades performance. The 4-layer and wide 512/1024-class models consistently rank in the bottom half.
3. **Over-parameterisation does not help**: `mlp_2x1024` (1.06M params) performs worse than `mlp_3x64` (9K params), confirming the bottleneck is the loss function, not model capacity.
4. **x–y asymmetry**: All models show $|x \text{ error}| \gg |y \text{ error}|$, reflecting the dominant B_y field component that drives bending in the x–z plane.

Figure 3 compares all models across the key error metrics as a heatmap.

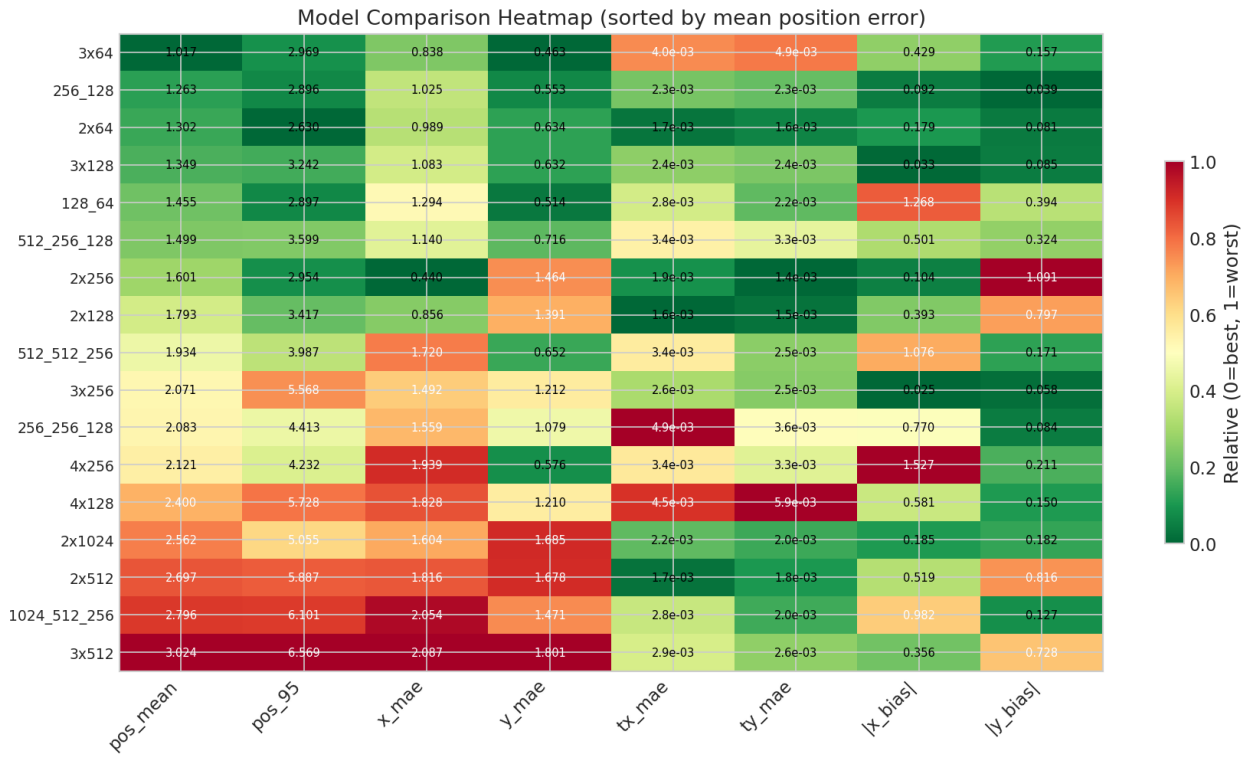

Figure 3: Multi-model comparison heatmap showing position and slope errors across all 17 architectures.

4 Diagnostic Analysis: Why V1 Performance Saturates

4.1 MSE Contribution Breakdown

For the best model (`mlp_3x64`), the contribution of each output component to the total MSE loss is:

$$\text{MSE}_x : \text{MSE}_y : \text{MSE}_{t_x} : \text{MSE}_{t_y} \;=\; 78.4\% : 21.6\% : 0.0\% : 0.0\% \tag{3}$$

The slopes t_x and t_y contribute *negligibly* to the total loss. This means the optimiser effectively ignores bending physics — the model learns only the kinematic mapping $x' \approx x + t_x \cdot \Delta z$ and does not invest capacity in the field-driven slope changes Δt_x and Δt_y.

4.2 Target Dynamic Range

The root cause is the extreme dynamic range between output components (Table 2).

Table 2: Standard deviation of raw output targets vs. residual (field-driven) targets, showing the compression ratio. The residual formulation removes the dominant kinematic term.

Component	Raw σ	Residual σ	Compression
x_{out} / δx_{field}	827.1564	0.8397	985×
y_{out} / δy_{field}	689.9211	0.0868	7951×
t'_x / Δt_x	0.1733	0.000189	919×
t'_y / Δt_y	0.1444	0.0000255	5656×

The raw x_{out} has standard deviation $\sigma = 827\,\text{mm}$, while the field-driven deflection $\delta x = x' - (x + t_x \cdot \Delta z)$ has $\sigma = 0.84\,\text{mm}$ — a compression of 985×. The model wastes the vast majority of its capacity learning the trivial linear kinematic term.

Figure 4 visualises the x–y deflection asymmetry and its connection to the dipole field structure.

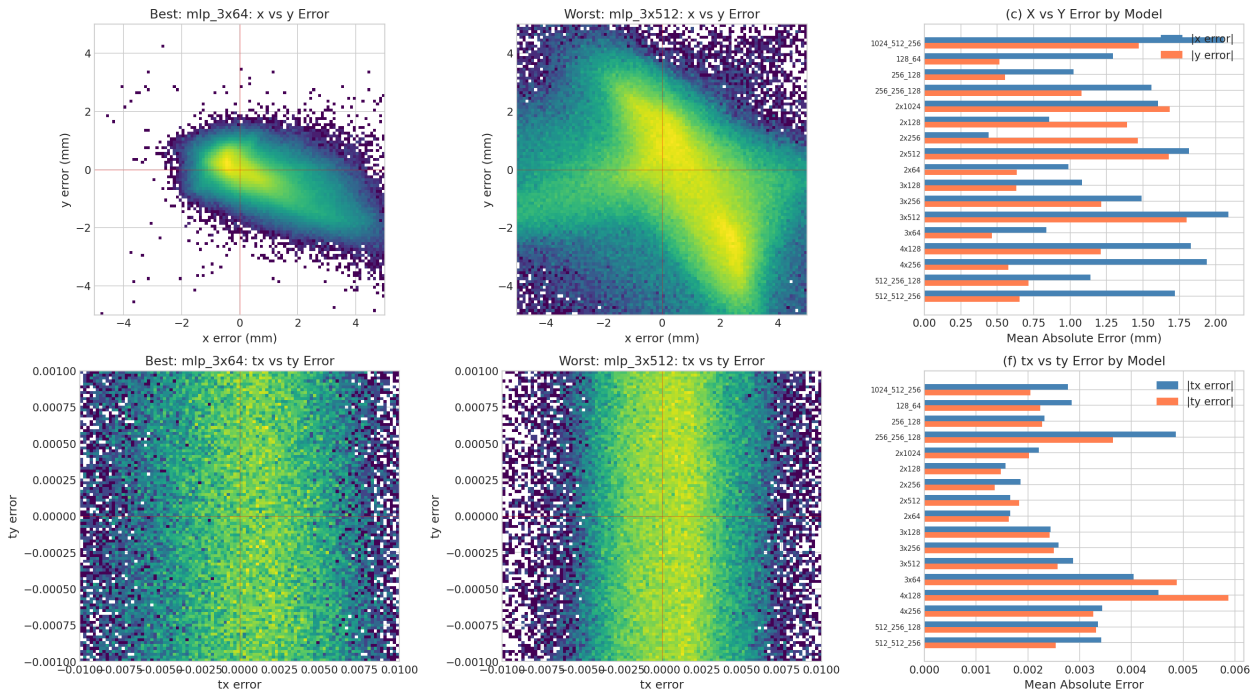

Figure 4: Position error asymmetry between x (bending plane) and y (non-bending plane) for the best model. The x deflection is an order of magnitude larger, reflecting the dominant B_y component.

4.3 Momentum Dependence

Table 3 shows the residual target standard deviations by momentum bin. Low-momentum tracks ($p < 2\,\text{GeV}$) have $\sim$27× larger $\sigma(\delta x)$ than high-momentum tracks ($p > 20\,\text{GeV}$). The log-uniform sampling ensures equal representation, but the variance-dominated loss still focuses on low-p tracks.

Table 3: Residual target statistics by momentum bin (500K test samples). Deflections scale as $\Delta t_x \propto q/p$, so low-p tracks dominate the variance.

Momentum bin	N	$\sigma(\delta x)$ (mm)	$\sigma(\delta y)$ (mm)	$\sigma(\Delta t_x)$	$\sigma(\Delta t_y)$
1–2 GeV	75 283	1.8741	0.1942	0.000421	0.000057
2–5 GeV	99 466	0.8616	0.0880	0.000193	0.000026
5–20 GeV	151 140	0.2985	0.0310	0.000067	0.000009
20–100 GeV	174 111	0.0702	0.0073	0.000016	0.000002

Figure 5 shows the error dependence on momentum, confirming the expected $1/p$ scaling.

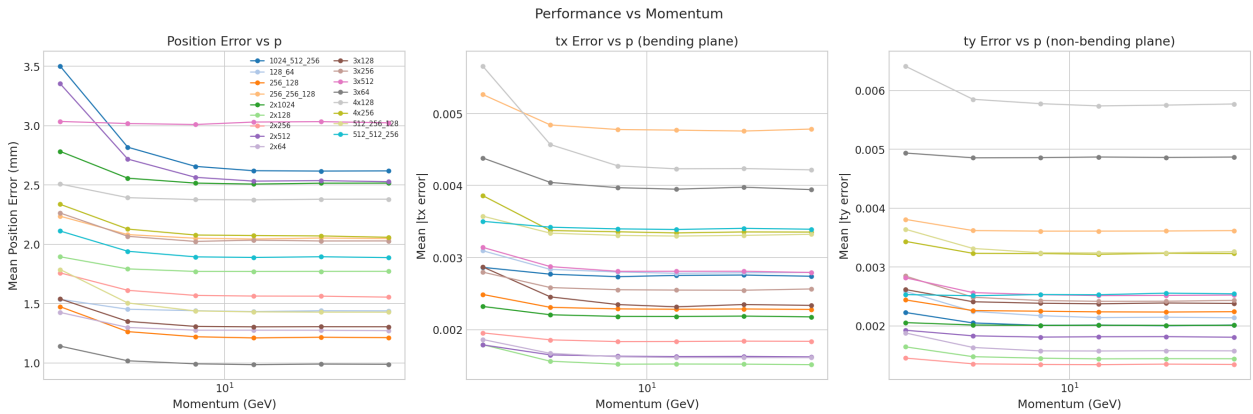

Figure 5: Position and slope errors as a function of track momentum. Low-momentum tracks ($p < 5\,\text{GeV}$) show significantly larger errors due to stronger field-driven deflection.

4.4 Spatial Dependence

Figures 6–8 show the error dependence on initial track state parameters and the $(z, \Delta z)$ phase space.

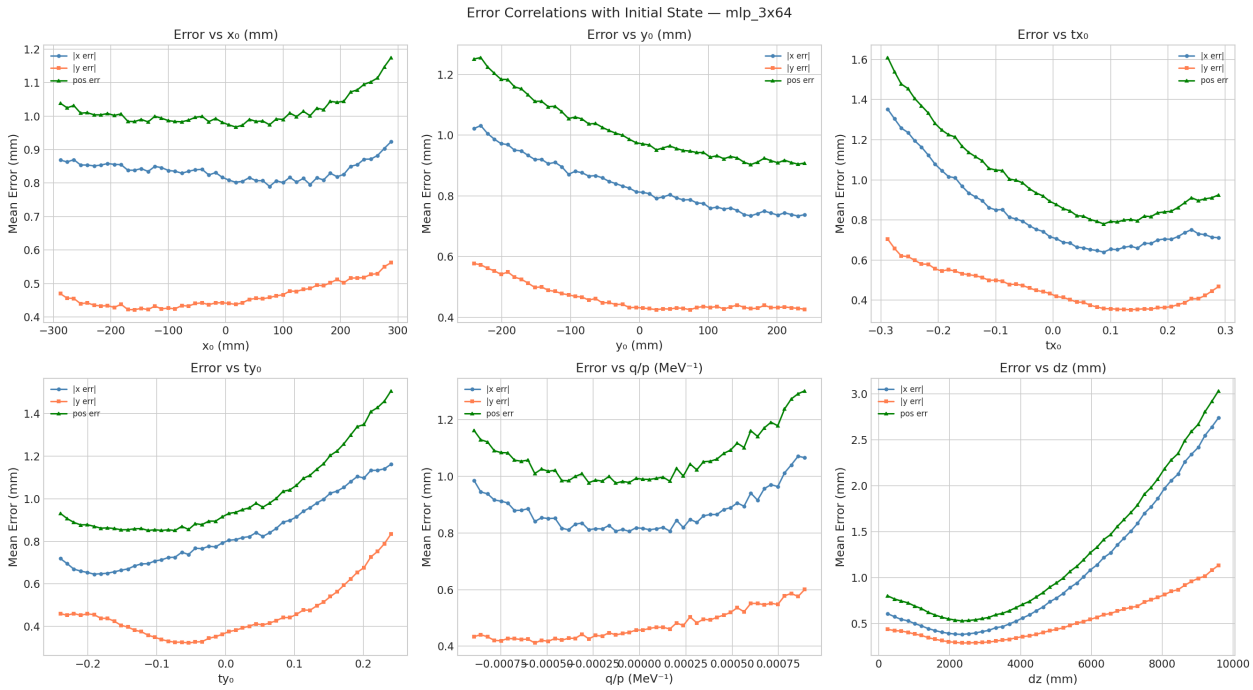

Figure 6: Position error as a function of initial track state parameters (x, y, t_x, t_y, q/p, Δz).

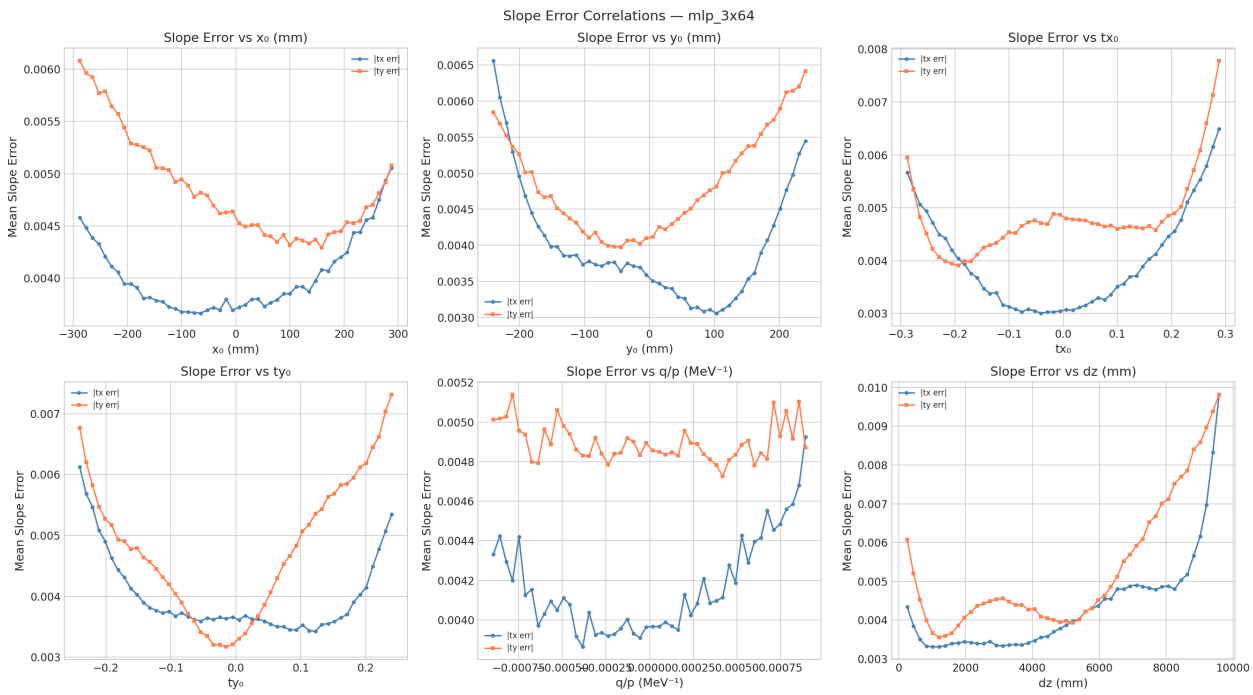

Figure 7: Slope error as a function of initial track state parameters.

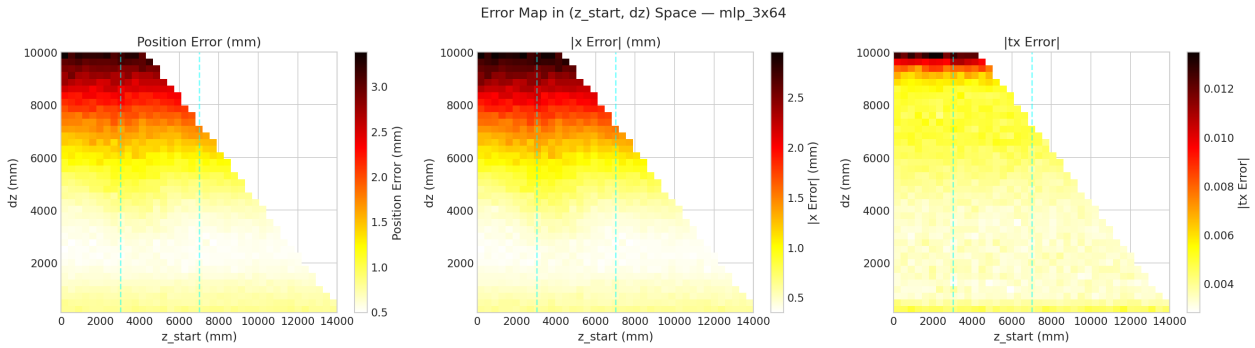

Figure 8: Position error heatmap in the $(z_{\text{start}}, \Delta z)$ plane. The error is largest for long-range propagations through the field centre ($z \approx 5000\,\text{mm}$, field peak).

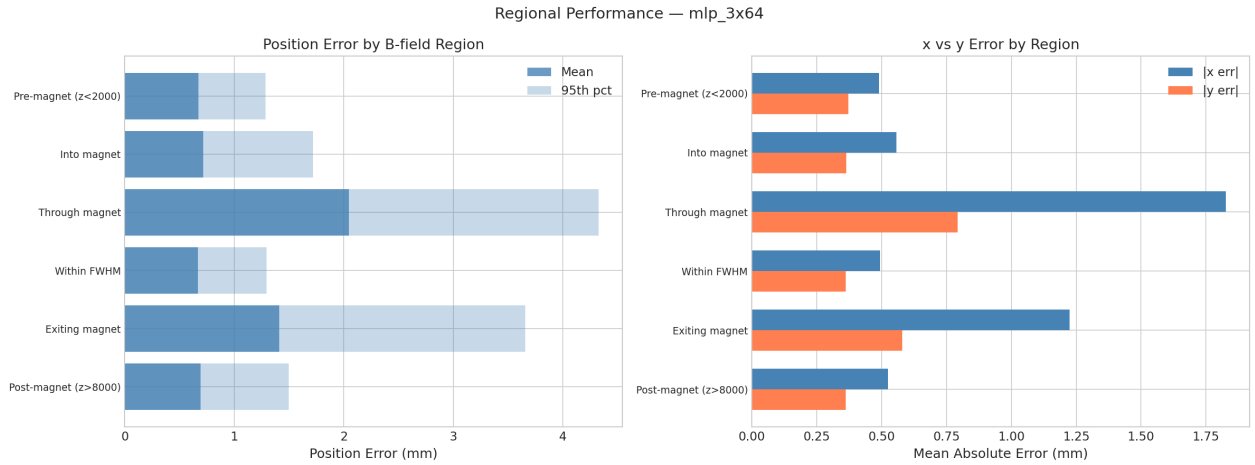

Figure 9: Regional performance breakdown showing error variation across different detector regions.

4.5 Error Distributions

Figure 10 shows the full error distributions for the best model.

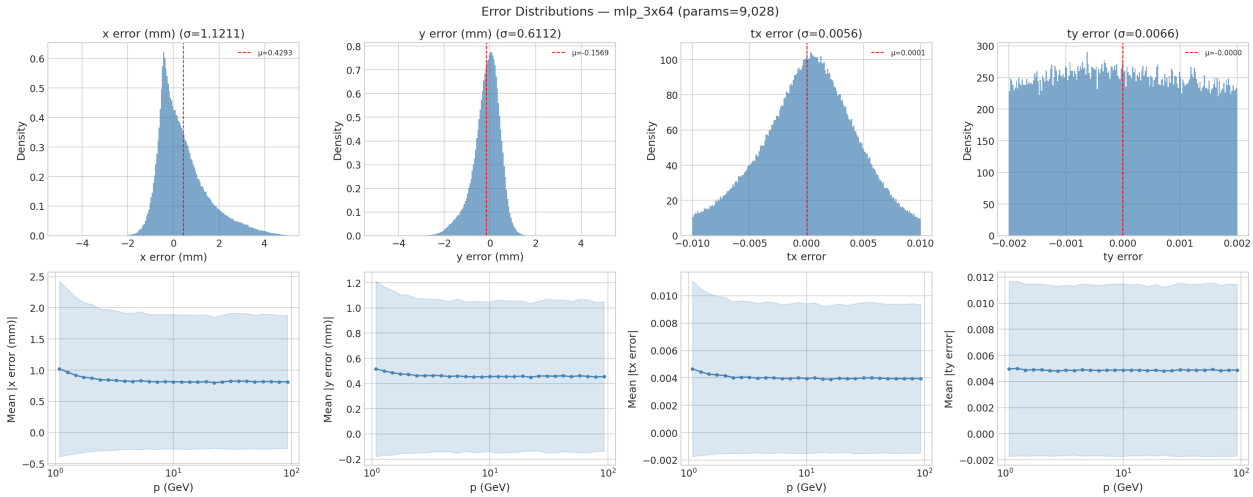

Figure 10: Detailed error distributions for the best model (`mlp_3x64`). The position error distribution has a long tail extending to $\sim 10\,\mathrm{mm}$ for extreme tracks.

5 Cost Function Weights for V2 Training

5.1 Residual Target Formulation

Following the analysis of Section 4.2, V2 training should use **residual targets** that remove the kinematic term:

$$\delta x = x' - (x + t_x \cdot \Delta z) \qquad \text{field-driven position deflection} \tag{4}$$

$$\delta y = y' - (y + t_y \cdot \Delta z) \qquad \text{field-driven position deflection} \tag{5}$$

$$\Delta t_x = t'_x - t_x \qquad \text{slope change (bending plane)} \tag{6}$$

$$\Delta t_y = t'_y - t_y \qquad \text{slope change (non-bending plane)} \tag{7}$$

The network then predicts corrections $(\hat{\delta x}, \hat{\delta y}, \widehat{\Delta t_x}, \widehat{\Delta t_y})$ and the full output is reconstructed as:

$$x' = x + t_x \cdot \Delta z + \hat{\delta x}, \qquad t'_x = t_x + \widehat{\Delta t_x}, \qquad \text{etc.} \tag{8}$$

5.2 Inverse-Variance Weighting

With residual targets, the component-wise variances measured on the test set are:

Table 4: Residual target variances and derived inverse-variance weights. The normalised weights sum to 4 (one per component).

Target	σ_i	σ_i^2	$1/\sigma_i^2$	w_i (norm.)
δx	0.839727	0.70514131	1.42	0.0000
δy	0.086769	0.00752889	1.33×10^2	0.0000
Δt_x	0.000189	0.00000004	2.81×10^7	0.0720
Δt_y	0.000026	0.00000000	1.54×10^9	3.9280

The raw inverse-variance weights span 9 orders of magnitude. After normalisation (weights summing to 4), nearly all weight is assigned to Δt_y (3.93) and Δt_x (0.07), with position components receiving negligible weight.

5.3 Practical Weight Prescription

The pure inverse-variance weights are extreme and would cause the model to entirely ignore position errors. In practice, several approaches are viable:

Option A: Log-balanced weights. Use logarithmically-spaced weights that partially compensate the dynamic range without fully inverting it:

$$\mathcal{L}_A = \mathrm{MSE}(\delta x) + 10\,\mathrm{MSE}(\delta y) + 10^4\,\mathrm{MSE}(\Delta t_x) + 10^6\,\mathrm{MSE}(\Delta t_y) \tag{9}$$

Option B: Standardised targets. Normalise each residual target to unit variance during preprocessing, then use uniform MSE:

$$\mathcal{L}_B = \mathrm{MSE}\left(\frac{\delta x}{\sigma_{\delta x}}\right) + \mathrm{MSE}\left(\frac{\delta y}{\sigma_{\delta y}}\right) + \mathrm{MSE}\left(\frac{\Delta t_x}{\sigma_{\Delta t_x}}\right) + \mathrm{MSE}\left(\frac{\Delta t_y}{\sigma_{\Delta t_y}}\right) \tag{10}$$

This is mathematically equivalent to inverse-variance weighting but numerically better conditioned.

Option C: Tunable α-balanced weights. Interpolate between uniform and inverse-variance using a parameter $\alpha \in [0, 1]$:

$$w_i = \sigma_i^{-2\alpha}, \qquad \alpha = 0 \text{ (uniform)}, \quad \alpha = 1 \text{ (full inverse-variance)} \tag{11}$$

Recommendation. We recommend **Option B** (standardised targets) for V2. This is the simplest to implement — the existing z-score normalisation pipeline already computes per-component means and standard deviations — and ensures all four components contribute equally to the gradient from the first epoch. The effective weights are:

$$\boxed{w_{\delta x} = \frac{1}{0.8397^2} = 1.42, \quad w_{\delta y} = \frac{1}{0.0868^2} = 133, \quad w_{\Delta t_x} = \frac{1}{(1.88 \times 10^{-4})^2} = 2.81 \times 10^7, \quad w_{\Delta t_y} = \frac{1}{(2.55 \times 10^{-5})^2}} \tag{12}$$

5.4 Additional Architecture Recommendations

Based on the V1 analysis, V2 should also incorporate:

1. **Residual targets**: Predict field-driven corrections rather than absolute states (Section 5.1).
2. **Multiplicative architecture**: The bending deflection $\Delta t_x \propto q/p$, so use:

$$\widehat{\Delta t_x} = (q/p) \cdot g_\theta(t_x, t_y, x, y, z, \Delta z) \tag{13}$$

   This guarantees $\Delta t_x = 0$ for straight tracks ($q/p = 0$) by construction.
3. q/p **skip connection**: Since q/p is an exact invariant ($d(q/p)/dz = 0$), do not predict it — pass it through unchanged.
4. **Small architecture**: The V1 sweep shows that small models ($<$ 35K params) outperform larger ones. Start with `[64, 64, 64]` or `[256, 128]` and tune from there.

6 Best Model Detailed Performance

Table 5 summarises the per-component errors for the best V1 model (`mlp_3x64`) in both raw and residual space.

Table 5: Detailed error breakdown for `mlp_3x64` (9 028 parameters).

Metric	x	y	t_x	t_y
MAE	0.838	0.463	0.0040	0.0049
Bias	+0.429	−0.157	$+1 \times 10^{-4}$	-5×10^{-6}
Position (combined)				
Mean	1.017 mm		—	
95th percentile	2.969 mm		—	
99th percentile	4.303 mm		—	
Slope (combined)				
Mean	—		0.007058	

7 Summary

The V1 architecture sweep demonstrates that:

1. A 3-layer MLP with only 9K parameters (`mlp_3x64`) achieves $\sim$1.0 mm mean position accuracy for LHCb track extrapolation over variable $\Delta z \in [100, 10\,000]$ mm, outperforming all larger architectures up to 1M parameters.

2. The naive MSE loss on raw output states wastes >99% of gradient signal on the kinematic $x + t_x \cdot \Delta z$ term, leaving slope predictions essentially untrained.

3. Over-parameterisation (up to 1M parameters) provides no benefit, confirming that model capacity is not the bottleneck — the loss function is.

For V2 training, we recommend:

- Residual targets (δx, δy, Δt_x, Δt_y) to remove the kinematic baseline
- Per-component standardisation to unit variance (equivalent to inverse-variance weighting)
- Multiplicative q/p gating for slope predictions
- A small architecture (`[64, 64, 64]` or `[256, 128]`)

The derived cost function weights and all model binaries have been exported for integration with the Gaudi `TrackMLPExtrapolator` framework.